THE SWEET TRIALS

SQUID GAME EDITION

"Luck or Skill: Your Candy. Your Fate."

3-slot Squid Game candy dispenser | Two modes: Luck & Skill

1. Project Overview

THE SWEET TRIALS is a Squid Game-themed interactive candy dispenser. Players choose between two challenges to earn candy: Luck (virtual slot machine in the app) or Skill (LED memory challenge on the physical machine). Built on an ESP32 running MicroPython, connected to a smartphone via Bluetooth Low Energy (BLE).

2. Components

Component	Qty	Purpose
12 x LEDs	12	9 for skill game (3 slots x 3 colors: Blue, Red, Yellow); 3 as mode indicators
IR Sensors	3	Detect player's hand under each slot
Reed Switches	3	Detect spinning wheel position (Blue / Red / Yellow zone)
Magnets	13	Mounted on wheel — trigger reed switches as wheel spins
Servo Motors	3	Open/close candy dispenser gate for each slot
ESP32	1	Main controller — MicroPython, BLE, LED/servo logic
Power Module	1	Powers ESP32 + peripherals (5 V and 3.3 V rails)
NeoPixel Strip	1	Win (rainbow/green) and lose (red flash) animations
Passive Buzzer	1	Losing sound
Start Button	1	Physical button to begin LED shuffle in Skill mode
MIT App Inventor App	1	Slot machine UI + BLE commands to ESP32

3. Game Logic

3.1 LUCK MODE — Casino Slot Machine (App-Managed)

The slot machine runs and if ypu hit the jackpot of three in a row, It is NOT physically wired to the ESP32 but MIT app gives the command to dispenser.

Three slots spin through random symbols.

- Slots decelerate and stop. If all three show the same symbol —JACKPOT!
- The app enables the 'Claim Candy' button.
- Host presses 'Claim Candy' button. App sends DISPENSE_ALL via BLE to the ESP32.
- ESP32 activates all three servo motors candy drops from every slot.
- NeoPixel rainbow win animation plays. If no jackpot, game resets.

3.2 SKILL MODE — LED Memory Challenge

The player must observe rapidly shuffling colored LEDs and remember which slot showed their assigned color then reach under it after all lights cut out.

Step	What Happens
1 — Select Mode	Player picks SKILL MODE.
2 — Spin Wheel	Player physically spins the colour wheel (3 zones). Reed switch complete the circuit as the arrow has the magnet.
3 — Colour Assigned	ESP32 reads which reed switch has a magnet aligned (Blue / Red / Yellow). Sends COLOR_BLUE, COLOR_RED, or COLOR_YELLOW .
4 — Press Start	Player presses the physical Start Button on the machine. ESP32 begins the shuffle.
5 — LED Shuffle	All 9 slot LEDs cycle rapidly: Blue → Red → Yellow → Blue... Each slot cycles independently.
6 — Lights Out	After 6 seconds, the ESP32 secretly records which colour each slot was showing, then turns ALL LEDs off. The player must remember what they saw at that final instant.
7 — Choose Slot	Player places hand under the slot they believe showed their assigned colour. The IR sensor detects the hand (active LOW output).
8 — Judgement	CORRECT: servo dispenses candy + NeoPixel rainbow + WIN sent to app. WRONG: buzzer beeps + red NeoPixel flash + LOSE sent to app.
Timeout	If the player does not choose within 10 seconds: automatic loss, buzzer, lose animation.

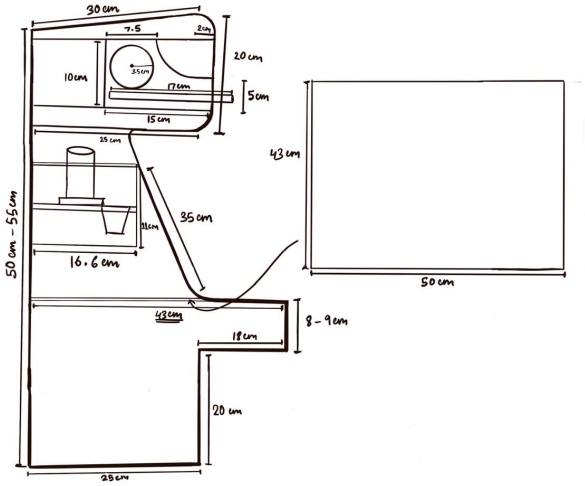
4. BLE Message Protocol

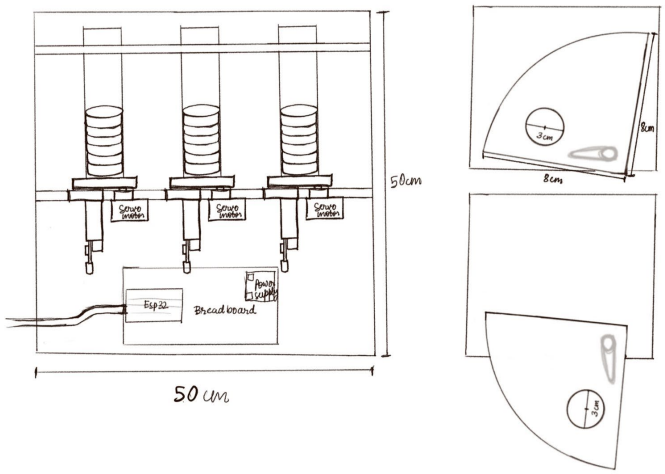
The app WRITES commands to the ESP32; the ESP32 NOTIFIES the app of game events. Both sides use the same UUIDs.

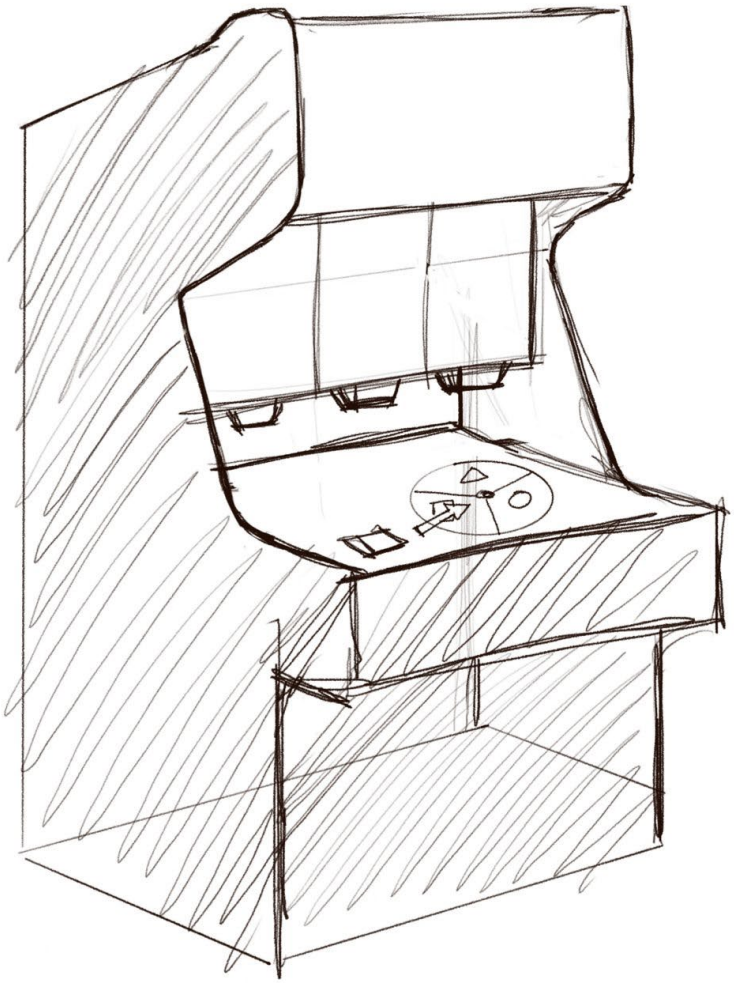
SERVICE_UUID: "12345678-1234-1234-1234-123456789abc"

CHAR_UUID: "abcdefab-1234-1234-1234-abcdefabcdef"

5. Sketches


30 cm
7.5
2cm
20 cm
10cm
3.5cm
17cm
5cm
15 cm
25 cm
43 cm
35 cm
11cm
16.6 cm
50 cm - 55cm
43cm
18cm
8 - 9cm
20 cm
25cm
50 cm


Servo motor
Servo motor
Servo motor
Esp32
Bread board
50cm
50 cm
3cm
8cm
8cm
3cm



6. Wiring Reference

All signal wires connect to the listed GPIO pins. Use 220 Ω–470 Ω resistors in series with all LEDs. Power servos and NeoPixel from the 5 V rail of the power module, not from the ESP32's 3.3 V pin.

GPIO	Connected To	Type	Notes
GPIO 2	Slot 1 — Blue LED	OUT	220 Ω series resistor
GPIO 4	Slot 1 — Red LED	OUT	220 Ω series resistor
GPIO 5	Slot 1 — Yellow LED	OUT	220 Ω series resistor
GPIO 18	Slot 2 — Blue LED	OUT	220 Ω series resistor
GPIO 19	Slot 2 — Red LED	OUT	220 Ω series resistor
GPIO 21	Slot 2 — Yellow LED	OUT	220 Ω series resistor
GPIO 22	Slot 3 — Blue LED	OUT	220 Ω series resistor
GPIO 23	Slot 3 — Red LED	OUT	220 Ω series resistor
GPIO 25	Slot 3 — Yellow LED	OUT	220 Ω series resistor
GPIO 26	Mode LED — Luck	OUT	220 Ω series resistor
GPIO 27	Mode LED — Skill	OUT	220 Ω series resistor
GPIO 32	Standby LED	OUT	220 Ω series resistor
GPIO 34	IR Sensor — Slot 1 OUT	IN	Input-only; 10 kΩ pull-down recommended
GPIO 35	IR Sensor — Slot 2 OUT	IN	Input-only; 10 kΩ pull-down recommended
GPIO 36	IR Sensor — Slot 3 OUT	IN	Input-only; 10 kΩ pull-down recommended
GPIO 39	Reed Switch — Blue Zone	IN	Input-only; no internal pull-up; 10 kΩ pull-down
GPIO 33	Reed Switch — Red Zone	IN	PULL_UP used in code
GPIO 13	Reed Switch — Yellow Zone	IN	PULL_UP used in code
GPIO 14	Servo 1 — Slot 1 signal	OUT	Servo power from 5 V rail
GPIO 15	Servo 2 — Slot 2 signal	OUT	Servo power from 5 V rail
GPIO 16	Servo 3 — Slot 3 signal	OUT	Servo power from 5 V rail
GPIO 17	NeoPixel DIN	OUT	300–500 Ω on data line; power from 5 V
GPIO 12	Buzzer +	OUT	Active buzzer: use Pin on/off. Passive: use PWM + tone
GPIO 0	Start Button	IN	PULL_UP; button connects GPIO 0 to GND

3.3 V	IR Sensor VCC	PWR	Check sensor datasheet — some need 5 V
5 V	Servos VCC, NeoPixel VCC	PWR	From power module — never from ESP32 5V pin directly
GND	All component GNDs	GND	Shared ground across all components

THE SWEET TRIALS —Good luck, and may the candy be in your favour.